

PyTorch & Deep Learning

Detailed Q&A Revision Guide

Nityaa Kalra | August 2026

SECTION 1: TENSORS & AUTOGRAD

Q1: What is a tensor in PyTorch?

A tensor is PyTorch's core data structure — essentially an n-dimensional array, similar to NumPy arrays but with GPU support and automatic differentiation. Everything in deep learning — weights, activations, gradients, inputs — is a tensor.

```
import torch # Create tensors x = torch.tensor([1.0, 2.0, 3.0]) x = torch.zeros(3, 4)
# 3x4 zeros x = torch.randn(batch, seq, hidden) # random normal # Key attributes
x.shape # torch.Size([3, 4]) x.dtype # torch.float32 x.device # cpu or cuda:0 # Move
to GPU x = x.to('cuda') x = x.cuda()
```

Q2: What is autograd and how does backpropagation work in PyTorch?

Autograd is PyTorch's automatic differentiation engine. When you create a tensor with `requires_grad=True` and run operations on it, PyTorch builds a computational graph. Calling `.backward()` on a scalar loss traverses this graph in reverse, computing gradients for every parameter using the chain rule.

```
x = torch.tensor([2.0], requires_grad=True) y = x ** 2 + 3 * x # forward pass -
builds graph y.backward() # backward pass - compute dy/dx print(x.grad) #
tensor([7.]) - dy/dx = 2x+3 at x=2 # In training: optimizer.zero_grad() # clear
previous gradients loss = criterion(output, labels) loss.backward() # compute
gradients optimizer.step() # update weights
```

Why `zero_grad()`? PyTorch accumulates gradients by default. If you don't zero them, gradients from the previous batch add to the current batch's gradients, corrupting training.

Q3: What is the difference between `.detach()` and `torch.no_grad()`?

`.detach()`: creates a new tensor that shares the same data but is detached from the computational graph. Use it to get a tensor value without tracking gradients for that tensor.

`torch.no_grad()`: context manager that disables gradient computation entirely for a block. Use during inference to save memory and speed up computation.

```
# Inference – no need to track gradients with torch.no_grad(): predictions =
model(input_ids) # Get numpy value for logging loss_value =
loss.detach().cpu().numpy()
```

SECTION 2: NEURAL NETWORK COMPONENTS

Q4: How do you build a neural network in PyTorch?

You subclass `nn.Module` and define layers in `__init__`, and the forward pass in `forward()`.

```
import torch.nn as nn class StanceClassifier(nn.Module): def __init__(self,
input_dim, hidden_dim, num_classes=3): super().__init__() self.fc1 =
nn.Linear(input_dim, hidden_dim) self.relu = nn.ReLU() self.dropout = nn.Dropout(0.3)
self.fc2 = nn.Linear(hidden_dim, num_classes) def forward(self, x): x = self.fc1(x) x
= self.relu(x) x = self.dropout(x) return self.fc2(x) # logits, not softmax model =
StanceClassifier(768, 256, num_classes=3) print(sum(p.numel() for p in
model.parameters())) # param count
```

Q5: What is the difference between `nn.CrossEntropyLoss` and `nn.NLLLoss`?

`nn.CrossEntropyLoss`: takes raw logits (no softmax applied) and internally applies `LogSoftmax` + `NLLLoss`. This is numerically more stable. Use this in almost all classification tasks.

`nn.NLLLoss`: takes log-probabilities (i.e., output of `LogSoftmax`). You'd use this if you manually apply `LogSoftmax` yourself.

```
# Correct – pass logits directly criterion = nn.CrossEntropyLoss() logits = model(x)
# shape: [batch, num_classes] loss = criterion(logits, labels) # labels: [batch] #
Weight classes for imbalanced data (your stance data is imbalanced!) weights =
torch.tensor([1.0, 2.5, 1.8]) # favor, against, none criterion =
nn.CrossEntropyLoss(weight=weights)
```

For your stance detection: class imbalance is a real issue (more 'none' examples). Using class weights in `CrossEntropyLoss` is a common fix.

Q6: What optimizers are common and when do you use each?

Optimizer	When to use	Key param
SGD	Large models, CV; needs tuning	lr, momentum=0.9
Adam	Default for NLP/fine-tuning	lr=1e-4, betas
AdamW	HuggingFace fine-tuning standard	lr, weight_decay
AdaFactor	Memory-efficient transformer training	scale_parameter=True

```
from torch.optim import AdamW optimizer = AdamW( model.parameters(), lr=2e-5, #
typical for fine-tuning transformers weight_decay=0.01 # L2 regularization )
```

Q7: What is a learning rate scheduler and why use it?

A scheduler adjusts the learning rate during training. Starting high and decaying helps: large steps early for fast convergence, small steps later for precision.

```
from transformers import get_linear_schedule_with_warmup # Warmup: lr gradually
increases from 0 to target for first N steps # Then linearly decays to 0 scheduler =
get_linear_schedule_with_warmup( optimizer, num_warmup_steps=100,
```

```
num_training_steps=total_steps ) # Standard PyTorch options scheduler =  
torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5) scheduler =  
torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=10)
```

SECTION 3: TRAINING LOOP

Q8: Write a complete PyTorch training loop.

This is the most important thing to know cold. Commit it to memory.

```
model = StanceClassifier(768, 256, 3) optimizer = AdamW(model.parameters(), lr=2e-5)
criterion = nn.CrossEntropyLoss() device = torch.device('cuda' if
torch.cuda.is_available() else 'cpu') model = model.to(device) for epoch in
range(num_epochs): # ■■ TRAIN ■■ model.train() total_loss = 0 for batch in
train_loader: input_ids = batch['input_ids'].to(device) labels =
batch['labels'].to(device) optimizer.zero_grad() # clear gradients logits =
model(input_ids) # forward pass loss = criterion(logits, labels) # compute loss
loss.backward() # backprop torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) #
grad clip optimizer.step() # update weights scheduler.step() # update lr total_loss
+= loss.item() # ■■ EVAL ■■ model.eval() with torch.no_grad(): for batch in
val_loader: logits = model(batch['input_ids'].to(device)) preds =
torch.argmax(logits, dim=-1) print(f'Epoch {epoch}:
loss={total_loss/len(train_loader):.4f}')
```

model.train() enables dropout and batch norm training mode. model.eval() disables them for inference. ALWAYS switch modes.

Q9: What is gradient clipping and why is it important?

Gradient clipping caps gradient magnitudes to prevent the exploding gradient problem — where gradients grow exponentially and cause unstable training (NaN losses, wild weight updates).

Particularly important for RNNs and long transformers.

```
# Clips gradients so their L2 norm never exceeds 1.0
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

SECTION 4: DATALOADER & DATASET

Q10: How does DataLoader work in PyTorch?

DataLoader wraps a Dataset object and handles batching, shuffling, and parallel data loading. You define HOW data is returned in Dataset.__getitem__; DataLoader handles the rest.

```
from torch.utils.data import Dataset, DataLoader class StanceDataset(Dataset): def
__init__(self, texts, labels, tokenizer, max_len=128): self.texts = texts self.labels
= labels self.tokenizer = tokenizer self.max_len = max_len def __len__(self): return
len(self.texts) def __getitem__(self, idx): encoding = self.tokenizer(
self.texts[idx], max_length=self.max_len, padding='max_length', truncation=True,
return_tensors='pt' ) return { 'input_ids': encoding['input_ids'].squeeze(),
'attention_mask': encoding['attention_mask'].squeeze(), 'labels':
torch.tensor(self.labels[idx], dtype=torch.long) } train_loader = DataLoader(
dataset, batch_size=16, shuffle=True, num_workers=4 )
```

SECTION 5: FINE-TUNING TRANSFORMERS

Q11: How do you fine-tune a pretrained HuggingFace model?

This is directly relevant to your Qwen3 fine-tuning work on Snellius.

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer from peft
import get_peft_model, LoraConfig, TaskType # Load pretrained model =
AutoModelForSequenceClassification.from_pretrained( 'Qwen/Qwen3-1.7B', num_labels=3
# favor, against, none ) # Add LoRA adapters (PEFT) lora_config = LoraConfig(
task_type=TaskType.SEQ_CLS, r=16, # rank of low-rank decomposition lora_alpha=32, #
scaling factor target_modules=['q_proj', 'v_proj'], lora_dropout=0.1, ) model =
get_peft_model(model, lora_config) model.print_trainable_parameters() # much less
than full fine-tune
```

Q12: What is LoRA and how does it reduce memory?

LoRA (Low-Rank Adaptation) adds small trainable matrices A and B to frozen weight matrices W. Instead of updating the full W (e.g., $4096 \times 4096 = 16\text{M}$ params), you update A ($4096 \times r$) and B ($r \times 4096$) where r is small (e.g., 16). The effective weight update is: $W' = W + AB$.

Memory reduction: $r=16 \rightarrow 2 \times (4096 \times 16) = 131\text{K}$ params vs 16M for full W. That's 100x fewer trainable parameters.

QLoRA additionally quantizes W to 4-bit, reducing its memory footprint by 8x vs float32. A and B remain in higher precision (bfloat16). You used this with Unsloth on Snellius.

Q13: What's the difference between model.train() and model.eval()?

model.train(): enables Dropout (randomly zeros activations) and BatchNorm tracking (uses batch statistics). These add regularization during training.

model.eval(): disables Dropout (all neurons active for full model capacity) and BatchNorm uses running statistics. Use during validation and inference.

Forgetting to call model.eval() before inference is a common bug that causes non-deterministic results.

SECTION 6: GPU & MEMORY

Q14: How do you handle GPU memory issues in PyTorch?

Common techniques:

1. Reduce batch size
2. Gradient accumulation — accumulate gradients over N steps before updating
3. Mixed precision training (fp16/bf16) — halves memory for activations
4. Gradient checkpointing — recompute activations during backward instead of storing
5. Use QLoRA/LoRA instead of full fine-tuning

```
from torch.cuda.amp import autocast, GradScaler # Mixed precision training
scaler = GradScaler()
with autocast(): # forward in fp16
    loss = criterion(model(x), y)
scaler.scale(loss).backward() # scale loss to avoid fp16 underflow
scaler.step(optimizer)
scaler.update() # Gradient accumulation
accumulation_steps = 4
for i, batch in enumerate(loader):
    loss = model(batch) / accumulation_steps
    loss.backward()
    if (i+1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

Q15: How do you save and load a PyTorch model?

Always save state_dict (just weights), not the whole model object — more portable.

```
# Save
torch.save(model.state_dict(), 'stance_model.pt') # Save checkpoint with training state
torch.save({'epoch': epoch, 'model_state_dict': model.state_dict(), 'optimizer_state_dict': optimizer.state_dict(), 'loss': loss, }, 'checkpoint.pt') # Load
model.load_state_dict(torch.load('stance_model.pt', map_location='cpu')) # Load checkpoint
checkpoint = torch.load('checkpoint.pt')
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
```